



Laboratorio #1

Programa: Tecnología en Sistemas.

Materia: Desarrollo de Software III.

Docente: Mg(c). Juan Pablo Pinillos Reina.

Yu-Gi-Oh! Duel Lite

Contexto

Vas a construir una mini-aplicación de escritorio en **Java Swing** que simula un duelo sencillo de Yu-Gi-Oh! entre un jugador y la máquina, utilizando datos obtenidos en vivo desde la **API YGOProDeck**.

Cada jugador tendrá **3 cartas Monster obtenidas aleatoriamente**, con imagen y atributos.

Por turnos, el jugador selecciona qué carta usar, mientras que la máquina elige la suya al azar. En cada turno solo habrá una carta ganadora según sus stats. El primero que gane **2 de 3 rondas** es el vencedor.

Objetivo de aprendizaje

- Consumir una **API REST** desde Java (JSON).
- Aplicar **POO** con clases claras y responsabilidades separadas.
- Usar **listeners** para la interacción en Swing.
- Integrar **UI + lógica de negocio** en un prototipo funcional.
- Trabajar en **parejas**, fortaleciendo la colaboración y práctica virtual.



Laboratorio #1

Programa: Tecnología en Sistemas.

Materia: Desarrollo de Software III.

Docente: Mg(c). Juan Pablo Pinillos Reina.

Requisitos funcionales (mínimos)

1. Selección de cartas (mazo inicial):

- Al iniciar, cada jugador recibe **3 cartas Monster** obtenidas desde la API (randomcard.php).
- Deben mostrarse en pantalla con **imagen, nombre, ATK y DEF**.

2. Vista de cada carta:

- Imagen oficial (URL proporcionada por la API).
- Nombre y valores de ATK / DEF.

3. Sistema de duelo simplificado:

- El **turno inicial** se define aleatoriamente.
- Cada turno:
 - El **jugador** elige una de sus cartas disponibles.
 - La **máquina** selecciona una al azar.
 - Se comparan **ATK vs DEF**:
 - Si ambos están en ataque → gana el mayor ATK.
 - Si uno en ataque y otro en defensa → comparar ATK del atacante contra DEF del defensor.
 - El ganador obtiene **1 punto de ronda**.
- El primero en ganar **2 rondas** es el **ganador del duelo**.

4. Interfaz de usuario:

- Mostrar las **cartas del jugador** con imagen y datos.
- Log de batalla desplazable (**JTextArea + JScrollPane**) con eventos:
 - Quién jugó qué carta.
 - Resultado del turno.
 - Puntaje acumulado.
- Anuncio del **ganador final**.



Laboratorio #1

Programa: Tecnología en Sistemas.

Materia: Desarrollo de Software III.

Docente: Mg(c). Juan Pablo Pinillos Reina.

Requisitos técnicos

- **Java 11+.**
- **Swing** para la interfaz.
- Consumo de API con **java.net.http.HttpClient** + librería ligera de JSON (ej.: org.json).
- Clases mínimas sugeridas:
 - Card (modelo: nombre, atk, def, imagen).
 - YgoApiClient (consulta y parseo JSON).
 - Duel (reglas y lógica del enfrentamiento).
 - BattleListener (eventos de la partida).

Endpoints de YGOProDeck

- Cartas al azar:

<https://db.ygoprodeck.com/api/v7/randomcard.php>

Nota: Validar que la carta obtenida sea tipo *Monster*. Si no, volver a solicitar.

Interacción y Listeners (obligatorio)

- **ActionListener** para botones: “Iniciar duelo”, “Elegir carta”.
- **BattleListener** para desacoplar lógica/UI. Debe notificar al menos:
 - onTurn(String playerCard, String aiCard, String winner)
 - onScoreChanged(int playerScore, int aiScore)
 - onDuelEnded(String winner)



Laboratorio #1

Programa: Tecnología en Sistemas.
Materia: Desarrollo de Software III.
Docente: Mg(c). Juan Pablo Pinillos Reina.

Validaciones y UX

- No iniciar el duelo hasta que ambos tengan sus **3 cartas cargadas**.
- Manejo visible de errores: “No se pudo cargar la carta”, “error de red”.

Restricciones

- No usar frameworks pesados (Spring, Retrofit, etc.).
- Prohibido bloquear el hilo de la UI durante peticiones/redibujado.
- Código con paquetes organizados y comentado.

Entregables

1. Repositorio con el código fuente (Maven/Gradle o proyecto IntelliJ).
2. README con:
 - Instrucciones de ejecución.
 - Breve explicación de diseño (1–2 párrafos).
 - Capturas de pantalla.

¡A duelar!   